LLMs, tools, and agents

Theory, then R versus Python

Bastien Chassagnol

Table of contents

1. Welcome

These notes are a **draft course** on large language models (LLMs). The first half is conceptual. The second half is practical: the same ideas in Python, then in R, then a side-by-side comparison.

The book layout follows the Quarto **book** pattern used in BAUDOT-lab/teaching_material (HTML site plus a downloadable PDF).

1.1. How to read this book

1. **Theory.** Tokens, next-token prediction, and why models hallucinate (LLM basics). Then three patterns that are often confused: MCP, RAG, and agents (Tools versus agents).
2. **Python.** Tool calling from a script, then a minimal MCP server you can attach to a host such as Cursor.
3. **R.** A condensed `ellmer` workshop, adapted from Hadley Wickham's A no bullshit guide to LLMs (useR! 2025 / Tidy Design, 2026), then R versus Python.

Live API calls are **not** evaluated at render time. Copy the chunks into your own session after you have set a provider key.

```
%%{init: {'theme': 'sandstone'}}%%
flowchart TD
  T["Theory: how LLMs work"] --> P["Patterns: RAG, MCP, agents"]
  P --> Py["Python: tools and MCP"]
```

```
  P --> R["R: ellmer chat, structure, tools"]
  Py --> C["R versus Python"]
  R --> C
```

Figure 1.1.: Course pipeline: theory, then two implementation tracks, then a comparison.

1.2. Build locally

From the repository root:

```
quarto render          # HTML and PDF under docs/
quarto preview         # live HTML preview
```

Requirements: Quarto 1.8 or newer, plus a LaTeX engine (`tinytex` or TeX Live) for the PDF.

1.3. Repository map

```
.
├── theory/                # conceptual chapters
├── python/
│   ├── llm-tool-calling/  # lab: provider SDK + tools
│   └── basic-mcp-server/  # lab: tiny MCP server
├── R/
│   └── llm-integration/   # lab: ellmer scripts
├── figures/               # shared images
└── docs/                  # rendered HTML and PDF
```

Part I.

Theory

2. LLM basics

This chapter is a compact map of how large language models behave in practice. It is not a substitute for a deep-learning textbook. The goal is enough vocabulary to read the rest of the course without treating the model as magic.

2.1. What an LLM actually does

A modern LLM is a **next-token predictor**. Given a sequence of tokens (pieces of text), it assigns probabilities to possible next tokens and samples from that distribution. Training on a huge corpus makes the distribution rich enough that the same mechanism can summarise, translate, write code, or refuse a request — all as *more text*.

That framing has three immediate consequences:

- **There is no built-in world clock.** Unless a tool or a retrieval step injects facts, the model only has what is in the prompt plus what was frozen into the weights.
- **There is no built-in calculator.** Arithmetic and counting are not guaranteed; they are patterns the model may or may not reproduce. Hadley Wickham's counting examples (letters in a word, large multiplications) are the same point in comic form (A no bullshit guide to LLMs).

- **Answers look finished even when they are guesses.** Models are trained to continue fluently. They rarely volunteer "I do not know" unless the system prompt or the product layer forces it.

2.2. Tokens, context, and cost

Text is chopped into **tokens** (sub-word pieces). Context windows are measured in tokens, not characters. Everything you send — system instructions, conversation history, retrieved documents, tool schemas, images encoded as tokens — shares that budget.

Practical habits:

- Put stable instructions in a **system prompt** (tone, role, output schema). Put the variable task in the user turn.
- Prefer structured outputs (JSON / a typed schema) when you will parse the reply in code. See the R lab in LLM integration.
- Treat **cost** as tokens in plus tokens out. A five-dollar API credit is enough for a first course lab; it is not enough for an unbounded agent loop.

2.3. Training versus inference

Stage	What happens	What you control in this course
Pre-training	Predict the next token on web-scale text	Nothing (you use a hosted model)
Post-training	Instruction following, tool use, safety	Choose the model family

Stage	What happens	What you control in this course
Inference	Sample tokens for *your* prompt	Prompt, tools, retrieval, temperature

You will almost never fine-tune a foundation model here. You will **steer** one: prompts, tools, and retrieved context.

2.4. Hallucination is a feature of the objective

If the most likely continuation is a confident-looking citation that does not exist, the model may emit it. Grounding strategies do not make the sampler honest; they **change the prompt**:

- **RAG** inserts retrieved passages so the likely continuation is "according to this excerpt…" (next chapter).
- **Tools** let the model request a fact (the date, a SQL query) instead of inventing it.
- **Structured extraction** plus human or statistical checks treats the model as a noisy sensor, not as an oracle.

2.5. The jagged edge

Capability is uneven. The same model can draft a working regular expression and then miscount letters in `unconventional`. Product teams paper over embarrassing failure modes with extra prompt rules; those patches move, so **evals** on *your* task matter more than a leaderboard screenshot.

A useful mental split for coding (again after Wickham):

- **Translation** (clear spec in one form → another form) is often a win: `curl` to `httr2`, SQL to `dplyr`, a sketch to a Shiny skeleton.

- **Open-ended design** (vague goal, many constraints) is where agent loops shine *and* where they waste tokens.

2.6. Where we go next

The next chapter separates three architectures that marketing copy lumps together: **MCP** (how apps talk to tools), **RAG** (how a query fetches documents), and **agents** (a plan–act–observe loop). They compose; they are not alternatives to "using an LLM".

3. MCP, RAG, and AI agents

MCP, RAG, and agents are often sold as competing products. They are three **layers**. MCP is connectivity. RAG is retrieval. An agent is a control loop. One application can use all three.

The diagram below (ByteByteGo) is the course's reference picture. Read the three columns left to right: plumbing, then a single grounded answer, then an iterative worker.

3.1. MCP: a standard way to plug tools in

MCP is an open standard protocol. It connects AI models to external tools and data sources. These can be APIs, databases, or apps like Gmail, Slack, or GitHub. So instead of you writing the integration or doing the integration for each of these applications separately, MCP basically gives you a standard way to connect to these systems.

In Figure 3.1 the **host** (Claude Desktop, an IDE, an agent app) is an MCP **client**. It speaks one protocol to many MCP **servers**. Each server then does the proprietary work: call GitHub, query Postgres, read a folder.

That is why MCP shows up in this course after "tool calling" in Python. A tool is a function plus metadata. MCP is how that function is **discovered and invoked across process boundaries**, instead of being hard-wired into one notebook.

See the official specification at modelcontextprotocol.io. A minimal server lives in `python/basic-mcp-server/`.

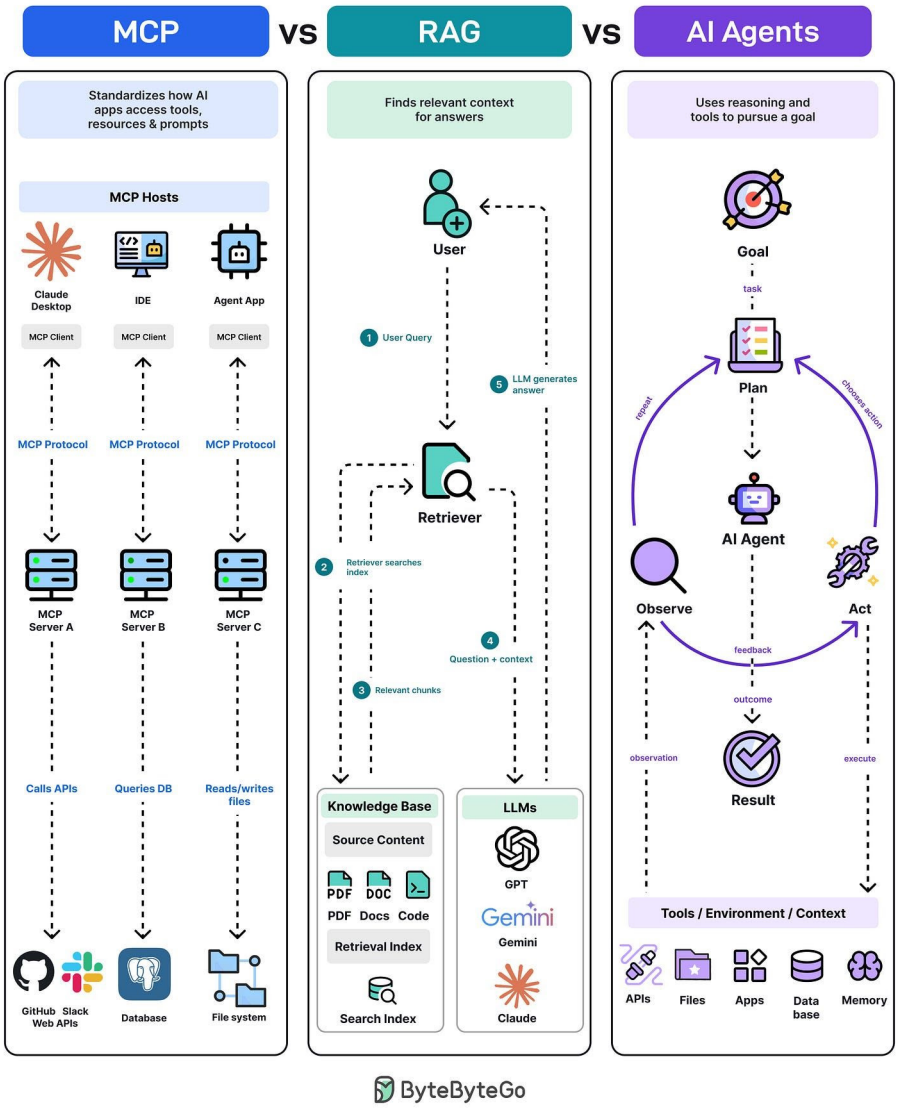

Figure 3.1.: Side-by-side architectures for MCP (hosts and servers), RAG (retrieve then generate), and AI agents (plan–act–observe).

```
%%{init: {'theme': 'sandstone'}}%%
flowchart TD
  H["MCP host: IDE, desktop, agent"] --> C["MCP client"]
  C -->|"MCP protocol"| SA["Server A: APIs"]
  C --> SB["Server B: database"]
  C --> SC["Server C: files"]
```

Figure 3.2.: One protocol, many servers: hosts stay interchangeable; integrations live behind MCP servers.

3.2. RAG: fetch context when the question arrives

In RAG, the model pulls the fresh information when a query comes. And this is why the model does not have to make stuff up on its own, but instead it fetches the fresh information from external data sources, like docs, PDFs, and databases, to give the most up-to-date answer to the prompt.

The numbered path in the middle of Figure 3.1 is a **request–response** pipeline:

1. The user asks a question.
2. A retriever searches an index (embeddings, lexical search, or both).
3. Relevant **chunks** come back from the knowledge base.
4. Question plus chunks go to the LLM.
5. The LLM writes an answer grounded in that context.

RAG does not, by itself, let the model send an email or delete a file. It changes **what is in the prompt**. If the index is stale, the answer is stale. If retrieval misses the right passage, the model may still invent a fluent gap-filler — which is why citations and chunk inspection belong in any serious RAG lab.

3.3. Agents: pursue a goal, not a single reply

An AI agent is kind of an AI system where the agent performs the task autonomously and takes the decisions. And then making sure everything is working fine, instead of a chatbot, which is really a request-response.

The right-hand column of Figure 3.1 is a loop: **goal → plan → act → observe → re-plan**, until a stopping condition. The "act" step is tool use against APIs, files, apps, databases, or memory.

Hadley Wickham's one-line definition is enough for the R lab: an agent is a chatbot with at least one **read** tool and one **write** tool (A no bullshit guide to LLMs). Listing files then deleting CSVs is already an agent — and already a security problem. Treat "YOLO" file deletion as a teaching example, not a production pattern.

```
%%{init: {'theme': 'sandstone'}}%%
flowchart LR
  G["Goal"] --> P["Plan"]
  P --> A["Act with tools"]
  A --> O["Observe"]
  O --> R{"Done?"}
  R -->|no| P
  R -->|yes| X["Result"]
```

Figure 3.3.: Agent control loop: choose an action, observe the environment, update the plan.

3.4. How to choose

Need	Pattern
The answer lives in *your* documents	RAG
The model must call *many* existing apps	MCP (or equivalent tool gateway)
The task needs several steps and checks	Agent loop
One shot: classify, extract, rewrite	Plain chat (maybe structured output)

Compose rather than pick a camp. A coding agent in Cursor is an agent host that speaks MCP to servers; it may also RAG over the repository. A RAG chatbot with no tools is still just request–response.

The Python chapters implement a **tool** and an **MCP server**. The R chapter implements the same ideas inside `ellmer`.

Part II.

Python

4. Python: LLM tool calling

This chapter is the Python counterpart of the R `ellmer` lab. A **tool** is still a function plus a JSON schema the model can request. The provider does **not** run your Python. It returns a tool call; your process executes it and sends the result back.

Runnable copy: `python/llm-tool-calling/`.

4.1. Setup

Use `uv` (see the project Python notes). Do not call `pip install` from a notebook that might run at check time.

```
cd python/llm-tool-calling
uv venv
uv add anthropic python-dotenv
export ANTHROPIC_API_KEY="…"   # or OPENAI_API_KEY
```

4.2. Chat, then a tool

The `today` tool is deliberately tiny: it exists because the model has no clock. The `multiply` tool exists because next-token prediction is a bad calculator.

4. Python: LLM tool calling

```python
# python/llm-tool-calling/tool_calling.py
from datetime import date

from anthropic import Anthropic

TOOLS = [
    {
        "name": "today",
        "description": "Return today's date in ISO format.",
        "input_schema": {
            "type": "object",
            "properties": {},
            "additionalProperties": False,
        },
    },
    {
        "name": "multiply",
        "description": "Multiply two numbers exactly.",
        "input_schema": {
            "type": "object",
            "properties": {
                "a": {"type": "number"},
                "b": {"type": "number"},
            },
            "required": ["a", "b"],
        },
    },
]


def run_tool(name: str, args: dict) -> str:
    if name == "today":
        return date.today().isoformat()
```

```
    if name == "multiply":
        return str(args["a"] * args["b"])
    raise ValueError(f"unknown tool: {name}")


def chat_with_tools(prompt: str) -> str:
    client = Anthropic()
    messages = [{"role": "user", "content": prompt}]
    while True:
        resp = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            tools=TOOLS,
            messages=messages,
        )
        if resp.stop_reason != "tool_use":
            return "".join(
                b.text for b in resp.content if b.type == "text"
            )
        tool_results = []
        for block in resp.content:
            if block.type != "tool_use":
                continue
            output = run_tool(block.name, block.input)
            tool_results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": output,
                }
            )
        messages.append({"role": "assistant", "content": resp.content})
        messages.append({"role": "user", "content": tool_results})
```

The `while` loop **is** the seed of an agent: observe a tool request, act locally, send the observation back. MCP later standardises how those tools are advertised to a host.

4.3. Try

```
print(chat_with_tools("What day is it today?"))
print(chat_with_tools("What is 384729 * 918273?"))
```

Compare with R: LLM integration: same round trip, different SDK. The comparison chapter (R versus Python) lists when you might wrap this loop in LangChain, or skip it and speak MCP instead.

5. Python: a minimal MCP server

MCP is the **host–server** contract from MCP, RAG, and agents. Your IDE (the host) already knows how to list tools, call them, and show the transcript. You write a **server** that exposes resources and tools.

This lab is intentionally small: one tool that echoes a greeting, one that returns the working directory. Production servers add auth, timeouts, and allow-lists.

Project files: `python/basic-mcp-server/`.

5.1. Run

```
cd python/basic-mcp-server
uv venv
uv add mcp
uv run python server.py
```

Point a host at the server over **stdio** (the usual desktop pattern). In Cursor, add an MCP server entry whose command is `uv run python server.py` with this folder as the working directory. The host then lists `hello` and `pwd` like any other tool.

5.2. Server sketch

```python
# python/basic-mcp-server/server.py
from pathlib import Path

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("course-demo")


@mcp.tool()
def hello(name: str = "world") -> str:
    """Return a short greeting. Useful as a smoke test for the host."""
    return f"Hello, {name}."


@mcp.tool()
def pwd() -> str:
    """Return the server process working directory."""
    return str(Path.cwd())


if __name__ == "__main__":
    mcp.run()
```

5.3. What this is not

- Not RAG: there is no index and no chunking.
- Not a full agent: the **host** decides the loop; this process only answers tool calls.

- Not a replacement for the Anthropic/OpenAI SDK lab. MCP shines when **several apps** must share the same tools without each notebook reimplementing GitHub, Slack, or Gmail.

Next: the same mental model in R with `ellmer` (LLM integration).

Part III.

R

6. R: LLM integration with ellmer

This lab is a **short** R track. It compresses Hadley Wickham's A no bullshit guide to LLMs (talk at useR! 2025; written up on Tidy Design, September 2026). Read that essay for tone and stories. Here we keep only the mechanisms you will type.

Package docs: `ellmer`. Scripts: `R/llm-integration/`.

> **API keys and render**
>
> Chunks are `eval: false`. Put a provider key in the environment (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, or Gemini's free-tier credentials). Wickham's advice for a first session: a few dollars of credit, or Gemini's free tier — not a local model until you already have a working cloud baseline.

6.1. Why bother, given that models "kind of suck"

Wickham's opening is the right lab safety briefing:

- Results are **stochastic**; the same prompt can count letters three different ways.
- Models **change** under you (product patches, not sudden intelligence).
- The **jagged edge**: fluent and wrong on easy tasks; surprisingly good on messy language.

- Outputs look **complete**. Doubt is rare unless you demand a schema or a tool.

And yet the same sampler writes a limerick, extracts a table from photos, and drives a quiz. The course stance is **yes, and**: treat the model as a biased, noisy function, then wrap it with structure and tools.

6.2. 1. Chat objects

`ellmer` is chat in R without copying from a browser. A **system prompt** sets tone for the whole object (terse, quiz-master, JSON only). You cannot always do that in a consumer web UI.

```
# R/llm-integration/01_chat.R
library(ellmer)

chat <- chat_anthropic(
  system_prompt = "Be terse. Prefer R examples."
)
chat$chat("Who are you?")
```

Switch provider with `chat_openai()`, `chat_google_gemini()`, and friends. The object stores history; later turns see earlier ones.

6.3. 2. Structured data: turn mess into rectangles

Data scientists already work with rectangles. The high-leverage move is to **force a schema** (animal name, background colour, …) so the reply is a data frame, not a paragraph. Images go in with `content_image_file()`. Variation across repeats is expected; kitten versus cat is often a labelling problem, not a bug.

Once you have a table, ordinary statistics apply: measurement error is assumed, not a scandal. For systematic checks, Wickham points to Simon Couch's `vitals` package (evals, in the LLM sense).

```
# Sketch — check current ellmer helpers before class
# (APIs move; see https://ellmer.tidyverse.org/)
chat <- chat_anthropic()
# chat$chat_structured(
#   content_image_file("animals/kitten.jpg"),
#   type = type_object(
#     animal = type_string("Common name of the animal."),
#     background = type_string("Dominant background colour.")
#   )
# )
```

6.4. 3. Tools: function plus metadata

The model has no clock and must not invent one. A **tool** is an R function plus a description (and argument types). The provider never runs R on its servers: it returns "please call `today`"; `ellmer` runs `Sys.Date()` on **your** machine and sends the string back.

```
# R/llm-integration/02_tools.R
library(ellmer)

chat <- chat_anthropic()
chat$register_tool(tool(
  fun = \( ) as.character(Sys.Date()),
  name = "today",
  description = "Return today's date (ISO)."
))
chat$register_tool(tool(
```

```
  fun = \(a, b) a * b,
  name = "multiply",
  description = "Multiply two numbers exactly.",
  arguments = list(
    a = type_number(),
    b = type_number()
  )
))
chat$chat("What day is it? What is 384729 * 918273?")
```

That is the same round trip as Python tool calling.

6.5. 4. Agents, in one line

Wickham: **agent = chatbot + a read tool + a write tool**. `list.files()` plus `file.remove()` is enough to delete every CSV in the working directory after the model *looks*. That is the demonstration and the warning. Guardrails today are thin ("YOLO"). Do not register destructive tools on a shared project folder.

The theory chapter (MCP, RAG, and agents) places this loop next to MCP and RAG so it is not confused with "chat that retrieves PDFs".

6.6. 5. Coding with LLMs

Four product shapes, increasing in coupling:

1. Web chat (copy-paste; goes stale like Excel → Word).
2. Inline autocomplete (skill: ignore the bad completions).
3. In-IDE chat (same model, more file context).
4. **Agentic coding** (Cursor, Claude Code, …): write, test, iterate.

Two **clear wins** Wickham emphasises — keep these as lab exercises:

- **Translation** when the spec is already complete: `curl` → `httr2`, SQL → `dplyr`, LaTeX → Quarto, a UI sketch → a Shiny first draft. Iterate when the first draft is wrong (his penguins app).
- **Lower activation energy**: a bad first draft you are willing to fix; a four-year `testthat` issue you finally throw an agent at; a domain you do not know yet. Claude's line, quoted in the essay: models make the cost of *trying* low enough that you attempt work you would otherwise postpone.

6.7. Concerns, without the TED-talk padding

From the same essay, stripped to lab policy:

- **Access:** start with a small API budget or Gemini's free tier.
- **Environment:** individual chat queries are tiny next to a flight; still do not run idle agent loops for fun.
- **Privacy:** do not paste confidential patient or unpublished data into a consumer endpoint. Institutions already negotiate cloud contracts; you personally may not have those terms.
- **Power:** the essay's "evil billionaires" aside is political, not a package API. For the course: know who hosts the weights and what they log.

Wickham's closer is the course closer too: one small experiment beats doomscrolling. Models are not about to replace a working statistician who can specify a schema, read a residual plot, and refuse a bad tool.

6.8. Try this session

1. `chat$chat()` with a terse system prompt.
2. Register `today` and `multiply`; ask a combined question.

3. Optional: structured extraction on two images; bind rows; notice disagreement.
4. Do **not** register `rm` unless you are in a throwaway temp directory (`withr::local_tempdir()`).

7. R versus Python

Same model, different glue. This chapter is a map, not a winner.

7.1. What stays the same

- Next-token sampling, tool round-trips, RAG, MCP, and agent loops are **language-agnostic**.
- You still owe the model a schema if you want a rectangle.
- Secrets live in the environment, never in the repo.

7.2. What usually differs

Job	R (this course)	Python (this course)
Chat + tools	`ellmer`	Anthropic / OpenAI SDK (see `tool_calling.py`)
Structured extract	`ellmer` types → tibble	JSON schema / Pydantic
Evals	`vitals`	Inspect, custom pytest
MCP server	Possible, less common in class	`mcp` FastMCP lab

Job	R (this course)	Python (this course)
RAG over PDFs	`ragnar`, `tidyllm`, or call Python	LangChain / LlamaIndex / plain embeddings
Agent host	Positron / Cursor calling R	Cursor / Claude Code (often Python-first)
Rectangles after extract	`dplyr` / modelling as usual	`pandas` / `polars`

R shines when the **downstream** work is statistical and the LLM is a parser. Python shines when you need the MCP ecosystem, vector stores, or the same language as the agent host.

7.3. Suggested pairing in class

1. Implement `today` + `multiply` in both languages. Diff the transcripts.
2. Expose `pwd` via MCP (Python) and call it from Cursor; then register an equivalent tool only inside `ellmer` (no MCP). Notice which tools the **IDE** can see.
3. Optional RAG mini-lab (either language): chunk this book's `theory/` folder, ask a question that is answered only in the MCP section of MCP, RAG, and agents, and inspect retrieved chunks.

7.4. Further reading

- Hadley Wickham, A no bullshit guide to LLMs
- Model Context Protocol

- ellmer
- ByteByteGo figure in MCP, RAG, and agents